Service mesh and Kubernetes networking

Understand pod networking, CNI and network policy, then run a service mesh with mTLS, traffic routing and troubleshooting.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. Kubernetes networking under the hood (14 min)
2. CNI, network policy and eBPF (13 min)
3. What a service mesh is and how it works (14 min)
4. mTLS and traffic management (15 min)
5. Observing and troubleshooting the mesh (14 min)
6. Sidecar and ambient mesh data planes (12 min)
7. Ingress gateways and the Gateway API (12 min)
8. Traffic shifting, retries and timeouts (13 min)
9. Mesh authorization and zero trust policy (13 min)
10. Multi-cluster mesh and failover (12 min)
11. Mesh performance, cost and resource tuning (12 min)
12. Operating and upgrading a service mesh (12 min)
Cheat sheet

Lesson 1. Kubernetes networking under the hood

14 minutes. Follow a packet from one pod to another: the pod network, CNI, Services, kube-proxy, CoreDNS, and how Ingress and the Gateway API bring traffic in.

Kubernetes promises that every pod can reach every other pod by IP address, without NAT. Something has to make that true. Knowing what that something is turns a mysterious timeout into a short list of places to look.

What you will learn

- The four rules of the Kubernetes network model
- What a CNI plugin does
- How a Service, kube-proxy and endpoints turn a name into a pod
- How DNS works inside a cluster
- Ingress and the Gateway API for traffic coming from outside

The model

1. Every pod gets its own IP address.
2. Pods can reach all other pods without NAT.
3. Agents on a node can reach all pods on that node.
4. A pod sees itself with the same IP address others see.

The cluster does not implement this itself. A **CNI plugin** (Container Network Interface), such as Calico, Cilium or Flannel, runs when a pod starts. It creates a virtual network device for the pod, assigns an address from the node's pod range and sets up the routes or tunnels that let other nodes reach it.

From a Service name to a pod

Pods come and go, so their addresses change. A **Service** gives a stable virtual IP (the ClusterIP) and a name. The pieces:

- The Service has a **selector**. Pods whose labels match become its **endpoints**.
- **kube-proxy** on each node programs rules (iptables or IPVS) that rewrite traffic sent to the ClusterIP so it goes to one of the endpoint pod IPs.
- **CoreDNS** answers `web.shop.svc.cluster.local` with the ClusterIP.

```
$ kubectl get svc,endpoints web -n shop
NAME          TYPE        CLUSTER-IP     PORT(S)
service/web   ClusterIP   10.96.14.20    80/TCP

NAME            ENDPOINTS                         AGE
endpoints/web   10.244.1.7:8080,10.244.2.4:8080   3d
```

If the **ENDPOINTS** column is empty, the selector matches no ready pods. That is the most common cause of "the Service does not work".

DNS in the cluster

Pods get `/etc/resolv.conf` pointing at CoreDNS and a search path such as `shop.svc.cluster.local svc.cluster.local cluster.local`. So `web` resolves inside the `shop` namespace, and `web.other` reaches another namespace. A setting called `ndots:5` makes short names try the search path first, which can slow lookups for external names.

Coming in from outside

MARGA by EknathaLabs marga.eknathalabs.com

- A **NodePort** opens a port on every node. A **LoadBalancer** Service asks the cloud (or MetalLB on bare metal) for an external IP.
- **Ingress** routes HTTP by host and path to Services, run by an ingress controller such as NGINX.
- The **Gateway API** is the newer, more expressive model: a `Gateway` for the listener, `HTTPRoute` for rules, with roles split between platform and application teams.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Kubernetes networking under the hood fits together

Part	What it does	What goes wrong
The model	Every pod gets its own IP address.	Blaming the app when the Service has no endpoints.
From a Service name to a pod	Pods come and go, so their addresses change.	Forgetting that the selector and labels must match exactly.
DNS in the cluster	Pods get /etc/resolv.conf pointing at CoreDNS and a search path such as shop.svc.cluster.local svc.cluster.local cluster.local.	Assuming DNS is always fast inside a cluster.
Coming in from outside	A NodePort opens a port on every node. A LoadBalancer Service asks the cloud (or MetalLB on bare metal) for an external IP.	Using NodePort in production without a load balancer in front.

Flow: The model to From a Service name to a pod / From a Service name to a pod to DNS in the cluster / DNS in the cluster to Coming in from outside

Incident: DNS slowed everything down

Incident: *Calls to an external API took five seconds, but only from pods.*

```
$ kubectl exec web-7d -- cat /etc/resolv.conf
search shop.svc.cluster.local svc.cluster.local cluster.local
nameserver 10.96.0.10
options ndots:5
```

The name `api.partner.com` has two dots, fewer than five, so the resolver tried each search suffix first and waited for failures. Ending the name with a dot (`api.partner.com.`) or lowering `ndots` cut the time to milliseconds.

Common mistakes

- Blaming the app when the Service has no endpoints
- Forgetting that the selector and labels must match exactly
- Assuming DNS is always fast inside a cluster
- Using NodePort in production without a load balancer in front

Check yourself

What gives a Service its list of pods?

Answer: Its label selector, which creates the endpoints

What does a CNI plugin do?

Answer: Gives each pod a network device and address and sets up routes between nodes

A Service shows no endpoints. What is the most likely cause?

Answer: The selector matches no ready pods

Why can external names be slow inside a pod?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: ndots:5 makes the resolver try search suffixes first

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. CNI, network policy and eBPF

13 minutes. Choose a CNI, understand what a NetworkPolicy allows and denies, and see where eBPF changes how Kubernetes networking works.

By default every pod can talk to every other pod. That is convenient for a small cluster and risky for a shared one. Network policies and the right CNI let you say who may talk to whom.

What you will learn

- How CNIs differ: overlay, routed, eBPF
- What a NetworkPolicy does and the default-deny pattern
- Why a policy does nothing without support from the CNI
- What eBPF changes in Cilium
- How to test a policy

Choosing a CNI

- **Flannel**: simple overlay networking, no policy support.
- **Calico**: routed or overlay networking with strong policy and BGP options.
- **Cilium**: built on **eBPF**, with policy, observability and optional replacement of kube-proxy.
- Cloud CNIs (AWS VPC CNI, Azure CNI) give pods addresses from the cloud network.

An **overlay** wraps pod traffic in tunnels (VXLAN) between nodes, which is easy but adds a small overhead. **Routed** designs advertise pod ranges to the network, which is faster and easier to debug with ordinary tools.

What a NetworkPolicy says

A NetworkPolicy selects pods and lists the traffic **allowed** to or from them. Policies are additive: if no policy selects a pod, everything is allowed; once any policy selects it for a direction, only what is listed in some policy is allowed.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny
  namespace: shop
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: web-from-ingress
  namespace: shop
spec:
  podSelector:
    matchLabels:
      app: web
  policyTypes: [Ingress]
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          name: ingress-nginx
    ports:
    - port: 8080
```

The first policy selects every pod and allows nothing: **default deny**. The second allows only the ingress controller to reach `web` on 8080. Remember to allow DNS (port 53 to CoreDNS) when you deny egress, or lookups break.

MARGA by EknathaLabs
marga.eknathalabs.com

Policy needs enforcement

A NetworkPolicy is only a rule. The **CNI** has to enforce it. Applying one on a cluster running Flannel is accepted without error and does nothing, which is a dangerous silent failure.

What eBPF changes

eBPF lets small, checked programs run inside the Linux kernel. Cilium uses it to:

- Enforce policy by identity (labels), not by IP address
- Replace long iptables chains with fast hash lookups, which matters with thousands of Services
- Show who talks to whom, with tools like Hubble

```
$ cilium status --brief
OK
$ kubectl exec -n shop deploy/web -- curl -s -m 2 http://db.shop:5432
curl: (28) Connection timed out after 2001 milliseconds
```

A timeout where you expect a refusal often means a policy is silently dropping the packets.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How CNI, network policy and eBPF fits together

Part	What it does	What goes wrong
Choosing a CNI	Flannel: simple overlay networking, no policy support.	Using IP addresses in policy instead of labels.
What a NetworkPolicy says	A NetworkPolicy selects pods and lists the traffic allowed to or from them.	Applying default-deny without first learning the real traffic.
Policy needs enforcement	A NetworkPolicy is only a rule. The CNI has to enforce it.	Assuming a policy is enforced without testing it.
What eBPF changes	eBPF lets small, checked programs run inside the Linux kernel.	Denying egress without allowing DNS.

Flow: Choosing a CNI to What a NetworkPolicy says / What a NetworkPolicy says to Policy needs enforcement / Policy needs enforcement to What eBPF changes

Incident: The policy that did nothing

Incident: *A security review found the database reachable from any namespace, despite a NetworkPolicy.*

```
$ kubectl get cm -n kube-system | grep -E "flannel|calico|cilium"
kube-flannel-cfg   2   200d
```

The cluster used plain Flannel, which does not enforce NetworkPolicy. The policy objects existed, but nothing applied them. The team moved to a CNI with enforcement and added a test that tries a blocked connection in CI.

Common mistakes

- Assuming a policy is enforced without testing it
- Denying egress without allowing DNS
- Using IP addresses in policy instead of labels
- Applying default-deny without first learning the real traffic

Check yourself

If no NetworkPolicy selects a pod, what traffic is allowed?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: All traffic

What can make a NetworkPolicy do nothing?

Answer: A CNI that does not enforce policies, such as plain Flannel

Why allow DNS when you deny egress?

Answer: Without it pods cannot resolve names and calls fail

What does Cilium use to enforce policy by identity?

Answer: eBPF programs in the kernel

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. What a service mesh is and how it works

14 minutes. Understand the problem a mesh solves, the sidecar and ambient models, the control plane and data plane, and when you should not use one.

When a company has hundreds of services, every one needs the same things: encrypted calls, retries, timeouts, metrics, access rules. A service mesh moves those jobs out of the application code and into the platform.

What you will learn

- The problem a mesh solves, and what it costs
- Data plane and control plane
- Sidecar proxies, and the newer ambient mode
- The main meshes: Istio, Linkerd, Cilium
- How to decide whether you need one

The problem

Without a mesh, each team writes its own retry logic, its own TLS setup and its own metrics. They differ by language, and a fix means redeploying every service. With a mesh, a proxy beside each service handles the network side the same way for everyone.

Two planes

- The **data plane** is the proxies that carry the real traffic. In Istio the proxy is **Envoy**.
- The **control plane** (Istio's `istiod`) is the brain. It watches Kubernetes and your mesh configuration, turns them into proxy configuration and pushes it out. It also issues the certificates used for mTLS.

```
$ kubectl get pods -n shop
NAME          READY   STATUS    RESTARTS
web-7d9f-x2   2/2     Running   0
$ kubectl get pod web-7d9f-x2 -n shop -o jsonpath='{.spec.containers[*].name}'
web istio-proxy
```

`2/2` ready means the pod runs the app and a proxy sidecar. Traffic in and out of the app is redirected through the proxy automatically.

Sidecar and ambient

- **Sidecar**: a proxy in every pod. It is simple to reason about and gives per-pod control, but it costs memory and CPU in every pod, and pods must be restarted to pick up proxy upgrades.
- **Ambient** (Istio): a shared node-level component handles encryption and basic policy (L4), and an optional per-namespace proxy handles HTTP features (L7). It lowers the per-pod cost.

Injection works through a label on the namespace (`istio-injection=enabled`). Pods created **before** the label do not get a sidecar until they restart.

The main meshes

Mesh	Style	Known for
Istio	Envoy, rich features	Powerful traffic control, larger to operate
Linkerd	Small Rust proxy	Simplicity, low overhead
Cilium	eBPF, sidecar-free option	Network integration

MARGA by EknathaLabs
marga.eknathalabs.com

When not to use one

A mesh adds moving parts, a new failure mode and a learning curve. If you have a handful of services, or need only TLS, a library or an ingress may be enough. Adopt a mesh for a clear need: mutual TLS everywhere, uniform telemetry, or fine traffic control.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How What a service mesh is and how it works fits together

Part	What it does	What goes wrong
The problem	Without a mesh, each team writes its own retry logic, its own TLS setup and its own metrics.	Adopting a mesh with no concrete problem to solve.
Two planes	The data plane is the proxies that carry the real traffic.	Upgrading the control plane with no staged rollout.
Sidecar and ambient	Sidecar: a proxy in every pod. It is simple to reason about and gives per-pod control, but it costs memory and CPU in every pod, and pods must be restarted to pick up...	Expecting existing pods to get sidecars without a restart.
The main meshes	.	Check the Common mistakes list below this diagram for what usually goes wrong here.
When not to use one	A mesh adds moving parts, a new failure mode and a learning curve.	Ignoring the extra CPU and memory the proxies need.

Flow: The problem to Two planes / Two planes to Sidecar and ambient / Sidecar and ambient to The main meshes / The main meshes to When not to use one

Incident: A mesh upgrade that took the checkout down

Incident: *After a control plane upgrade, some services returned 503 errors.*

```
$ istioctl proxy-status
NAME            CDS      LDS      EDS      RDS
checkout-1.shop STALE    SYNCED   SYNCED   SYNCED
```

Some proxies were stuck on old configuration because the control plane restarted before they reconnected. Rolling the control plane one replica at a time, and waiting for proxy-status to show SYNCED before each next step, avoided the problem on the next upgrade.

Common mistakes

- Adopting a mesh with no concrete problem to solve
- Expecting existing pods to get sidecars without a restart
- Ignoring the extra CPU and memory the proxies need
- Upgrading the control plane with no staged rollout

Check yourself

What carries real traffic in a service mesh?

Answer: The data plane: the proxies beside the services

What does the control plane do?

Answer: Turns your configuration into proxy settings, pushes them and issues certificates

You label a namespace for injection. Existing pods still show 1/1. Why?

Answer: Injection happens at pod creation, so they must be restarted

When might you avoid a service mesh?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: With very few services or when you only need simple TLS

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. mTLS and traffic management

15 minutes. Turn on mutual TLS, route traffic by weight for canaries, and set timeouts, retries and circuit breaking with VirtualService and DestinationRule.

Once the proxies are in place, the mesh gives you two big powers: every call can be encrypted and authenticated, and you can steer traffic with configuration instead of code.

What you will learn

- Mutual TLS, identity and the PeerAuthentication modes
- Authorisation policy
- VirtualService and DestinationRule for routing
- Canary releases with weights
- Timeouts, retries and circuit breaking, and how they can hurt

Mutual TLS

With ordinary TLS only the server proves who it is. In **mutual TLS (mTLS)** both sides do. The mesh gives each workload an identity (based on its service account), a short-lived certificate, and rotates it automatically.

`PeerAuthentication` sets the mode:

- **PERMISSIVE**: accepts both mTLS and plain text. Good while you migrate.
- **STRICT**: only mTLS. Plain text callers are rejected.

```
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: default
  namespace: shop
spec:
  mtls:
    mode: STRICT
```

An `AuthorizationPolicy` then says who may call what, for example only the `web` service account may call `GET /orders`.

Routing: VirtualService and DestinationRule

- A **DestinationRule** defines **subsets** of a service, usually by pod label (`version: v1`, `version: v2`), and policies such as connection limits.
- A **VirtualService** decides where requests go: by weight, by header, by path.

```
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: reviews
  namespace: shop
spec:
  host: reviews
  subsets:
  - name: v1
    labels:
      version: v1
  - name: v2
    labels:
      version: v2
---
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: reviews
```

MARGA by EknathaLabs marga.eknathalabs.com

```
  namespace: shop
spec:
  hosts:
  - reviews
  http:
  - route:
    - destination:
        host: reviews
        subset: v1
      weight: 90
    - destination:
        host: reviews
        subset: v2
      weight: 10
```

This sends 10 percent of calls to version 2: a **canary**. Raise the weight as metrics stay healthy. The weights in one route must add up to 100, and a subset named in a VirtualService must exist in a DestinationRule, or the proxy returns errors.

```
$ istioctl analyze -n shop
Error [IST0101] (VirtualService shop/reviews) Referenced host+subset in destinationrule not found: \
     "reviews+v2"
```

Timeouts, retries and circuit breaking

- A **timeout** stops a call from waiting forever.
- A **retry** repeats a failed call, which hides short faults but can multiply load.
- **Circuit breaking** (connection limits and outlier detection) stops sending traffic to a pod that keeps failing.

If service A retries three times and calls B, which retries three times and calls C, one slow C produces up to nine calls for each request: a **retry storm**. Put retries at one layer, use short per-try timeouts and cap the total.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How mTLS and traffic management fits together

Part	What it does	What goes wrong
Mutual TLS	With ordinary TLS only the server proves who it is.	Switching to STRICT before every caller has a sidecar.
Routing: VirtualService and	A DestinationRule defines subsets of a service, usually by pod label (version: v1, version: v2), and policies such as connection limits.	Forgetting that a VirtualService needs its subsets to exist.
Timeouts, retries and circuit	A timeout stops a call from waiting forever.	Weights that do not add up to 100.

Flow: Mutual TLS to Routing: VirtualService and / Routing: VirtualService and to Timeouts, retries and circuit

Incident: Retries made the outage worse

Incident: *A slow database caused a full outage of the storefront.*

Three layers each retried failed calls three times with no limit. When the database slowed, each user request became dozens of calls, which overloaded it further. Removing retries from the inner layers, adding per-try timeouts, and enabling outlier detection restored stability.

Common mistakes

- Switching to STRICT before every caller has a sidecar
- Weights that do not add up to 100

MARGA by EknathaLabs marga.eknathalabs.com

- Retries at several layers
- Forgetting that a VirtualService needs its subsets to exist

Check yourself

What is the difference between PERMISSIVE and STRICT mTLS?

Answer: PERMISSIVE accepts mTLS and plain text; STRICT accepts only mTLS

Where are subsets such as v1 and v2 defined?

Answer: In a DestinationRule

Why can nested retries be dangerous?

Answer: They multiply load and can turn a slowdown into an outage

What must the weights in a single route add up to?

Answer: 100

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. Observing and troubleshooting the mesh

14 minutes. Read mesh metrics and traces, check configuration sync with istioctl, and decode the common 503 and connection errors.

A mesh adds a hop to every call, which makes problems harder to place: is it the app, the proxy or the configuration? The same proxies that add the hop also give you the best visibility you will ever have.

What you will learn

- The metrics and traces a mesh produces for free
- Checking sync with `istioctl proxy-status` and `analyze`
- Reading proxy response flags in access logs
- A step-by-step method for a failing call

What you get for free

Every proxy reports the **golden signals** for each service pair: request rate, error rate, latency and saturation. Add distributed tracing headers and you can see a request's path through services. Dashboards such as Kiali (a topology graph), Grafana and Jaeger show it.

Is the configuration there?

Before blaming the app, check that the proxy has the configuration you wrote.

```
$ istioctl proxy-status
NAME             CLUSTER      CDS      LDS      EDS      RDS      ISTIOD
web-7d.shop      Kubernetes   SYNCED   SYNCED   SYNCED   SYNCED   istiod-6d4b9c7f5-xk2lp
reviews-9a.shop  Kubernetes   SYNCED   STALE    SYNCED   SYNCED   istiod-6d4b9c7f5-xk2lp
$ istioctl analyze -n shop
Info [IST0118] (Service shop/web) Port name  (port: 80, targetPort: 8080) doesn't follow the namin \
    g convention of Istio port.
```

- **STALE** or **NOT SENT** means the proxy has not received or accepted the latest configuration.
- `analyze` checks your configuration for mistakes. Port names that do not start with a protocol (`http-`, `grpc-`, `tcp-`) make the mesh treat the traffic as plain TCP, so HTTP features silently do not apply.

Reading response flags

Envoy access logs show a short **response flag** that says why a request failed.

Flag	Meaning	Look at
UF	Upstream connection failure	Is the destination pod up? mTLS mode mismatch?
UC	Upstream connection terminated	App closed the connection, timeouts
NR	No route configured	VirtualService host or Gateway
UH	No healthy upstream	Endpoints empty, readiness probes
URX	Retry limit exceeded	Backend failing; retries too high
DC	Downstream connection terminated	Client gave up

```
$ kubectl logs web-7d -c istio-proxy | tail -2
[2026-11-03T10:12:01.221Z] "GET /orders HTTP/1.1" 503 UH no_healthy_upstream - 0 19 0 - "-" "curl/ \
    8" "reviews.shop.svc.cluster.local:9080"
```

`503 UH` for the reviews service tells you no healthy endpoint exists, so check the selector, readiness and rollout of reviews.

A method for a failing call

MARGA by EknathaLabs marga.eknathalabs.com

1. **Which hop?** Use the trace or the access logs to find the first failing hop.
2. **Does the proxy have current config?** `istioctl proxy-status`.
3. **Is the config valid?** `istioctl analyze`.
4. **What does the flag say?** Read the response flag in the log.
5. **Does the destination have endpoints?** `kubectl get endpoints`.
6. **Is it a policy?** Check PeerAuthentication and AuthorizationPolicy.
7. **Take the mesh out of the picture.** Call the pod IP directly to see whether the app is the problem.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Observing and troubleshooting the mesh fits together

Part	What it does	What goes wrong
What you get for free	Every proxy reports the golden signals for each service pair: request rate, error rate, latency and saturation.	Looking at the app first when the proxy configuration is stale.
Is the configuration there?	Before blaming the app, check that the proxy has the configuration you wrote.	Port names without a protocol prefix.
Reading response flags	Envoy access logs show a short response flag that says why a request failed.	Ignoring response flags.
A method for a failing call	Which hop?. Use the trace or the access logs to find the first failing hop.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: What you get for free to Is the configuration there? / Is the configuration there? to Reading response flags / Reading response flags to A method for a failing call

Incident: Everything is up, nothing works

Incident: *The checkout page returned errors, but every pod was Running.*

`istioctl proxy-status` showed 40 proxies STALE. The control plane had run out of memory and restarted, and it could not push new configuration. Raising its memory limit and adding a second replica fixed it. Without proxy-status the team would have spent hours inspecting healthy pods.

Common mistakes

- Looking at the app first when the proxy configuration is stale
- Port names without a protocol prefix
- Ignoring response flags
- Running a single control plane replica

Check yourself

What does STALE mean in istioctl proxy-status?

Answer: The proxy has not received or accepted the latest configuration

In an Envoy log, what does UH mean?

Answer: No healthy upstream endpoints

Why should Service ports be named like http-web?

Answer: So the mesh knows the protocol and can apply HTTP features

A call fails with UF right after STRICT mTLS was set. What is the likely cause?

MARGA by EknathaLabs
marga.eknathalabs.com

Answer: A caller without a sidecar is calling in plain text

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. Sidecar and ambient mesh data planes

12 minutes. Compare the sidecar proxy model with the ambient, sidecar-free model and what each costs in resources, upgrades and isolation.

The data plane is the set of proxies that carry traffic. How they are deployed decides what the mesh costs and how risky it is to operate, so it is worth understanding both main designs.

What you will learn

- How sidecar injection works
- How ambient mode splits layer 4 and layer 7
- Overhead, upgrades and isolation trade-offs
- Choosing a model for your cluster

Sidecar model

A mutating webhook adds a proxy container to each pod. Traffic in and out is redirected through it, so it can encrypt, retry, route and measure every call.

```
kubectl label namespace shop istio-injection=enabled
kubectl rollout restart deployment -n shop
kubectl get pod -n shop -o jsonpath='{.items[0].spec.containers[*].name}'
```

The cost is one proxy per pod: extra memory, CPU, start-up time and a rolling restart of every workload to upgrade proxies. The benefit is strong isolation, because each workload has its own proxy and policy point.

Ambient model

In **ambient mode** a small per-node component handles encryption and basic layer 4 policy for all pods on that node. If you need layer 7 features such as header routing or retries, you add a **waypoint** proxy for a namespace or service.

```
kubectl label namespace shop istio.io/dataplane-mode=ambient
istioctl waypoint apply -n shop --enroll-namespace
```

Applications are not restarted to join, and the shared proxies are upgraded without touching pods. Trade-offs include a shared node component with a wider blast radius and fewer years of production experience than sidecars.

Choosing

Measure the real overhead of proxies in a test cluster. Use sidecars for strict per-workload isolation and mature features. Consider ambient for many small pods or when restarting workloads is costly. Whichever you choose, keep a plan for upgrading the data plane safely.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Sidecar and ambient mesh data planes fits together

Part	What it does	What goes wrong
Sidecar model	A mutating webhook adds a proxy container to each pod.	Forgetting that proxy upgrades need pod restarts in the sidecar model.
Ambient model	In ambient mode a small per-node component handles encryption and basic layer 4 policy for all pods on that node.	Using layer 7 features without a waypoint in ambient mode.
Choosing	Measure the real overhead of proxies in a test cluster.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Sidecar model to Ambient model / Ambient model to Choosing

Incident: Memory eaten by proxies

Incident: *Sidecar proxies used more memory than the applications they served.*

```
kubectl top pod --containers -n shop
app          38Mi
istio-proxy  96Mi
4200 pods in the cluster
```

Small services were wrapped with a default proxy that needed more memory than the service. The team tuned proxy limits for small workloads, scoped sidecar configuration to reduce memory use and moved low-risk namespaces to the shared node model after testing.

Common mistakes

- Enabling a mesh everywhere before measuring cost
- Forgetting that proxy upgrades need pod restarts in the sidecar model
- Using layer 7 features without a waypoint in ambient mode
- Ignoring the blast radius of a shared node component

Check yourself

What is a sidecar?

Answer: A proxy container in each pod

What does ambient mode remove?

Answer: Per-pod sidecar proxies

What provides layer 7 features in ambient mode?

Answer: A waypoint proxy

Why measure overhead first?

Answer: Proxies can cost more than small apps

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 7. Ingress gateways and the Gateway API

12 minutes. Bring outside traffic into a mesh with gateways, use the Kubernetes Gateway API and secure the edge with TLS and routing rules.

Traffic enters a cluster through a gateway before it meets the mesh. The older Ingress resource was limited, and the Gateway API replaces it with roles, richer routing and a standard that many implementations share.

What you will learn

- The roles of GatewayClass, Gateway and routes
- Routing by host, path and header
- TLS termination and certificate handling
- Timeouts, limits and safe change control

Resources

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: public
  namespace: infra
spec:
  gatewayClassName: istio
  listeners:
    - name: https
      protocol: HTTPS
      port: 443
      hostname: shop.example.com
      tls:
        mode: Terminate
        certificateRefs:
          - name: shop-cert
      allowedRoutes:
        namespaces:
          from: Selector
          selector:
            matchLabels:
              expose: "true"
```

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: shop-web
  namespace: shop
spec:
  parentRefs:
    - name: public
      namespace: infra
  hostnames: ["shop.example.com"]
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /api
      backendRefs:
        - name: api
          port: 8080
    - backendRefs:
        - name: web
          port: 80
```

The **GatewayClass** names the implementation, the **Gateway** defines listeners and is owned by the platform team, and **HTTPRoute** objects are owned by application teams and attach to the gateway. Selectors limit which namespaces may attach.

MARGA by EknathaLabs
marga.eknathalabs.com

TLS and limits

Terminate TLS at the gateway with certificates managed by an automated issuer, and keep mTLS inside the mesh. Set request timeouts, maximum body sizes and rate limits at the edge. Redirect HTTP to HTTPS and restrict allowed hosts.

Operating

```
kubectl get gateway,httproute -A
kubectl describe httproute shop-web -n shop
```

Read the route **status conditions**: they say whether it was accepted and which parent it attached to. Roll out route changes with review, and test a canary host before moving real traffic.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Ingress gateways and the Gateway API fits together

Part	What it does	What goes wrong
Resources	The GatewayClass names the implementation, the Gateway defines listeners and is owned by the platform team, and HTTPRoute objects are owned by application teams and...	One shared route object edited by many teams.
TLS and limits	Terminate TLS at the gateway with certificates managed by an automated issuer, and keep mTLS inside the mesh.	No timeouts or limits at the edge.
Operating	Read the route status conditions: they say whether it was accepted and which parent it attached to.	Not reading route status conditions.

Flow: Resources to TLS and limits / TLS and limits to Operating

Incident: One route hid another

Incident: *A new route with an overlapping prefix took traffic from the checkout service.*

```
HTTPRoute shop-web: Accepted=True
/api          -> api
/api/checkout -> checkout-v2 (backend has no such path)
checkout errors: 3% returned 404
```

A team added a more specific rule that pointed to a backend with a different path layout. Because the longest prefix wins, checkout traffic moved to it. The team added route tests in CI, reviews by the owning team and a canary hostname for new routes.

Common mistakes

- One shared route object edited by many teams
- Not reading route status conditions
- Terminating TLS without automatic renewal
- No timeouts or limits at the edge

Check yourself

Who usually owns the Gateway?

Answer: The platform team

What do HTTPRoute objects do?

Answer: Define routing rules for a gateway

MARGA by EknathaLabs
marga.eknathalabs.com

Where do you see why a route did not apply?

Answer: Status conditions

Why automate certificate renewal?

Answer: Expired certificates break the edge

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 8. Traffic shifting, retries and timeouts

13 minutes. Shift traffic gradually between versions, set safe retries and timeouts and avoid retry storms in a service mesh.

Mesh routing rules let you release gradually and add resilience without changing code. They also let you hurt yourself with a single setting, so you need to understand how weights, retries and timeouts combine.

What you will learn

- Weighted routing for canary releases
- Timeouts and retries that respect idempotency
- Circuit breaking and outlier detection
- Preventing retry storms

Weighted routing

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: api
spec:
  hosts: [api]
  http:
    - route:
        - destination: {host: api, subset: v1}
          weight: 95
        - destination: {host: api, subset: v2}
          weight: 5
      timeout: 3s
      retries:
        attempts: 2
        perTryTimeout: 1s
        retryOn: connect-failure,refused-stream,503
```

Start the canary at a small weight, compare error rate and latency of the two subsets, and increase in steps. Roll back by setting the weight to zero.

Timeouts and retries

Work backwards from the user's deadline. If the caller waits 3 seconds, three tries at 3 seconds each are pointless. Retry only **idempotent** operations or failures that happened before the request was processed, such as connection failures. Retrying a payment POST can charge twice.

Protecting the dependency

```
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: api
spec:
  host: api
  trafficPolicy:
    connectionPool:
      http: {http1MaxPendingRequests: 100, maxRequestsPerConnection: 50}
    outlierDetection:
      consecutive5xxErrors: 5
      interval: 10s
      baseEjectionTime: 30s
```

Connection limits act as circuit breakers by failing fast when a service is overloaded. **Outlier detection** ejects unhealthy endpoints for a time. Together with a **retry budget**, they stop callers from piling more load onto a failing service.

MARGA by EknathaLabs marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Traffic shifting, retries and timeouts fits together

Part	What it does	What goes wrong
Weighted routing	Start the canary at a small weight, compare error rate and latency of the two subsets, and increase in steps.	Per-try timeouts longer than the overall timeout.
Timeouts and retries	Work backwards from the user's deadline.	Retrying non-idempotent requests.
Protecting the dependency	Connection limits act as circuit breakers by failing fast when a service is overloaded.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Weighted routing to Timeouts and retries / Timeouts and retries to Protecting the dependency

Incident: Retries tripled the load

Incident: *A small slowdown became an outage because every layer retried.*

```
frontend -> api       attempts: 3
api -> db-proxy       attempts: 3
db-proxy requests per second: 9x normal
```

Each layer added its own retries. A brief slowdown at the database was multiplied by nine, which kept it overloaded. The team kept retries at one layer, added timeouts that fit inside the caller's deadline and set circuit breakers so overloaded services reject quickly.

Common mistakes

- Retrying at every layer
- Retrying non-idempotent requests
- Per-try timeouts longer than the overall timeout
- Moving canary weights without watching per-version errors

Check yourself

What should retries respect?

Answer: Idempotency and the caller's deadline

What does outlier detection do?

Answer: Ejects unhealthy endpoints temporarily

How should a canary start?

Answer: At a small weight with monitoring

What causes a retry storm?

Answer: Layers multiplying retries during a slowdown

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 9. Mesh authorization and zero trust policy

13 minutes. Use mTLS identities and authorization policies to allow only the calls you intend, with deny by default and safe rollout.

Encryption proves who is on the other end. Authorization decides what they may do. A mesh gives you both, replacing network location rules with identities that follow the workload.

What you will learn

- How mesh identities are issued and verified
- Strict mTLS and migration from permissive mode
- Writing allow policies and default deny
- Rolling out policy without breaking traffic

Identity and mTLS

Each workload gets a certificate tied to its Kubernetes **service account**, for example `spiffe://cluster.local/ns/shop/sa/checkout`. Start with **permissive** mode, which accepts both plain and mTLS traffic, then move to **strict** after you confirm every client uses the mesh.

```
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: default
  namespace: shop
spec:
  mtls:
    mode: STRICT
```

Allow policies

```
apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: payments-allow-checkout
  namespace: payments
spec:
  selector:
    matchLabels:
      app: payments
  action: ALLOW
  rules:
    - from:
        - source:
            principals: ["cluster.local/ns/shop/sa/checkout"]
      to:
        - operation:
            methods: ["POST"]
            paths: ["/charge"]
```

Once any ALLOW policy applies to a workload, requests that match no rule are denied. Add a namespace-wide default deny and allow each needed path explicitly.

Rollout

Inventory real traffic from mesh metrics first. Apply policies in audit or dry-run mode if your mesh supports it, watch denials in access logs and fix legitimate callers, then enforce. Keep policies in Git with owners and review, and alert on a sudden rise in denied requests.

```
kubectl get authorizationpolicy -A
kubectl logs -n payments deploy/payments -c istio-proxy | grep RBAC
```

MARGA by EknathaLabs
marga.eknathalabs.com

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Mesh authorization and zero trust policy fits together

Part	What it does	What goes wrong
Identity and mTLS	Each workload gets a certificate tied to its Kubernetes service account, for example spiffe://cluster.local/ns/shop/sa/checkout.	Switching to strict mTLS without finding non-mesh callers.
Allow policies	Once any ALLOW policy applies to a workload, requests that match no rule are denied.	Allow rules with wildcards for everything.
Rollout	Inventory real traffic from mesh metrics first.	Policies nobody owns or reviews.

Flow: Identity and mTLS to Allow policies / Allow policies to Rollout

Incident: Strict mode broke the batch job

Incident: *Enabling strict mTLS stopped a job that sent plain traffic.*

```
upstream connect error or disconnect/reset before headers
reset reason: connection termination
caller: batch-export (no sidecar)
```

The nightly export ran outside the mesh and sent plain traffic. When the namespace switched to strict, its requests were rejected. The team now inventories callers from mesh metrics before enforcing, adds exceptions with a deadline for legacy clients and tests policy changes in staging.

Common mistakes

- Switching to strict mTLS without finding non-mesh callers
- Allow rules with wildcards for everything
- No default deny
- Policies nobody owns or reviews

Check yourself

What identity does a workload present?

Answer: A certificate tied to its service account

What happens when an ALLOW policy exists and no rule matches?

Answer: The request is denied

What is permissive mTLS for?

Answer: Migration before strict mode

How do you find legitimate callers?

Answer: Mesh metrics and access logs

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 10. Multi-cluster mesh and failover

12 minutes. Connect services across clusters with shared trust, cross-cluster discovery and locality-aware failover, and know the risks.

Running more than one cluster gives you isolation and regional failover, but services still need to find and trust each other. A mesh can provide that, at the cost of more moving parts.

What you will learn

- Shared trust and identity across clusters
- Discovery and gateway patterns
- Locality-aware routing and failover
- Failure domains and capacity

Trust

Workloads in both clusters must trust the same **root certificate authority**, usually with a separate intermediate per cluster. Protect the root offline and rotate intermediates on a schedule. Without shared trust, mTLS between clusters fails.

Connectivity and discovery

There are two main shapes. In **flat** networking the pod networks are routable, so proxies connect directly. In **gateway-based** designs each cluster exposes an east-west gateway and traffic crosses it over mTLS. Each control plane or a replicated service registry learns about services in the other cluster.

```
istioctl remote-clusters
kubectl --context east get svc -n shop
kubectl --context west get endpoints api -n shop
```

Locality and failover

```
trafficPolicy:
  loadBalancer:
    localityLbSetting:
      enabled: true
      failover:
        - from: eu-west
          to: eu-central
  outlierDetection:
    consecutive5xxErrors: 5
    interval: 10s
    baseEjectionTime: 30s
```

Prefer local endpoints for latency and cost, and fail over only when local ones are unhealthy. Outlier detection is required for failover to trigger.

Independence

Give each cluster its own control plane if possible, and keep the system able to run if the link between them is cut. Plan spare capacity so one cluster can absorb the other's traffic, and run regular failover tests during working hours.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Multi-cluster mesh and failover fits together

Part	What it does	What goes wrong
Trust	Workloads in both clusters must trust the same root certificate authority, usually with a separate intermediate per cluster.	Assuming failover works without testing it.
Connectivity and discovery	There are two main shapes. In flat networking the pod networks are routable, so proxies connect directly.	Sizing each cluster only for its own traffic.
Locality and failover	Prefer local endpoints for latency and cost, and fail over only when local ones are unhealthy.	Locality and failover Prefer local endpoints for latency and cost, and fail over only when local ones are unhealthy.
Independence	Give each cluster its own control plane if possible, and keep the system able to run if the link between them is cut.	Cross-region traffic on every call.

Flow: Trust to Connectivity and discovery / Connectivity and discovery to Locality and failover / Locality and failover to Independence

Incident: Failover overloaded the survivor

Incident: *Traffic moved to the second cluster and overloaded it.*

```
eu-west: all endpoints unhealthy
eu-central: cpu 97%, p99 latency 4.2s
503 rate rising
```

The mesh failed over exactly as configured, but the second cluster was sized for half the traffic. The team sized each region to carry the other at peak, added autoscaling headroom and scheduled a quarterly failover test.

Common mistakes

- Assuming failover works without testing it
- Sizing each cluster only for its own traffic
- Sharing one point of failure such as a single root key
- Cross-region traffic on every call

Check yourself

What must clusters share for mTLS across them?

Answer: A trusted root of trust

What is an east-west gateway?

Answer: A gateway for cluster-to-cluster traffic

What triggers locality failover?

Answer: Unhealthy local endpoints

What must you plan for failover?

Answer: Spare capacity in the other cluster

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 11. Mesh performance, cost and resource tuning

12 minutes. Measure the latency and resource cost of a mesh, scope proxy configuration and tune limits and telemetry to keep it affordable.

A mesh is software in the request path, so it needs capacity planning like any other service. Most surprises come from proxies holding too much configuration or from too many metric labels.

What you will learn

- Measuring added latency and resource use
- Scoping proxy configuration to what a workload needs
- Reducing telemetry cost
- Sizing the control plane and proxies

Measure

Run the same load test against a service with and without the mesh, and compare p50 and p99 latency, CPU and memory. Typical sidecar overhead is small per request, but it adds up across many hops and many pods.

```
kubectl top pod -n shop --containers
istioctl proxy-status
istioctl proxy-config clusters deploy/web -n shop | wc -l
```

The last line counts how many clusters, meaning upstream services, the proxy holds. A large number means high memory.

Scope configuration

```
apiVersion: networking.istio.io/v1
kind: Sidecar
metadata:
  name: default
  namespace: shop
spec:
  egress:
    - hosts:
        - "./*"
        - "platform/*"
```

By default each proxy learns about every service in the mesh. A **Sidecar** resource limits it to the namespaces a workload really calls, which saves memory and speeds configuration pushes.

Telemetry

High-cardinality labels, such as user IDs or full paths, can explode metric series. Remove unneeded dimensions, sample traces and access logs, and keep detailed telemetry for important services only.

Sizing

Set requests and limits on proxies based on measurements, and size the control plane for the number of proxies and changes. Watch control plane push latency and rejected configuration, since slow pushes mean stale routing.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

Diagram: How Mesh performance, cost and resource tuning fits together

Part	What it does	What goes wrong
Measure	Run the same load test against a service with and without the mesh, and compare p50 and p99 latency, CPU and memory.	Never benchmarking with and without the mesh.
Scope configuration	By default each proxy learns about every service in the mesh.	Leaving every proxy with global configuration.
Telemetry	High-cardinality labels, such as user IDs or full paths, can explode metric series.	Keeping all metric labels.
Sizing	Set requests and limits on proxies based on measurements, and size the control plane for the number of proxies and changes.	Proxy limits copied from examples.

Flow: Measure to Scope configuration / Scope configuration to Telemetry / Telemetry to Sizing

Incident: Gigabytes of proxy memory

Incident: *Each proxy held configuration for every service in a large cluster.*

```
istio-proxy memory: 1.4Gi
clusters in config: 6900
pods OOMKilled in the last hour: 31
```

Nobody had scoped proxy configuration, so each sidecar tracked thousands of services. Proxies hit memory limits and restarted. A default Sidecar resource per namespace reduced configuration and memory sharply, and the team added a dashboard of proxy memory and config size.

Common mistakes

- Never benchmarking with and without the mesh
- Leaving every proxy with global configuration
- Keeping all metric labels
- Proxy limits copied from examples

Check yourself

What does a Sidecar resource limit?

Answer: What a proxy knows about

Why measure both p50 and p99?

Answer: Tail latency shows added hops

What causes metric cost to explode?

Answer: High-cardinality labels

What should size proxy limits?

Answer: Measured use

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 12. Operating and upgrading a service mesh

12 minutes. Upgrade a mesh with canary control planes, revision tags and staged proxy restarts, and know how to roll back and what to monitor.

The mesh sits in front of every request, so a bad upgrade can look like an outage everywhere. Staged, reversible upgrades and good health signals make it a routine operation.

What you will learn

- Version skew between control plane and proxies
- Canary upgrades with revisions
- Rolling proxies in waves and rolling back
- What to monitor during an upgrade

Revisions

```
istioctl install --set revision=1-24 -y
kubectl get pods -n istio-system -l app=istiod
kubectl label namespace shop istio.io/rev=1-24 --overwrite
kubectl rollout restart deployment -n shop
istioctl proxy-status
```

With **revisions** you install a second control plane beside the first. Each namespace points at one revision, so you move a test namespace first, then others. Proxies pick up the new version when their pods restart.

Skew and order

Control plane versions can usually be at most one minor version ahead of proxies. Upgrade the control plane first, then proxies, then gateways. Read release notes for removed APIs and changed defaults. Keep infrastructure code in Git so you can review the diff.

Rollback

Rolling back is relabeling the namespace to the old revision and restarting the workloads. Do not delete the old control plane until the new one has run healthy for a full business cycle.

Monitor

- Request success rate and p99 latency by service
- Control plane config push errors and rejected configuration
- Proxy and control plane restarts and memory
- Certificate issuance and expiry
- Gateway error rates

Define a go or no-go check for each wave, and schedule upgrades outside peak windows.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

Diagram: How Operating and upgrading a service mesh fits together

Part	What it does	What goes wrong
Revisions	With revisions you install a second control plane beside the first.	Not restarting workloads after changing revisions.
Skew and order	Control plane versions can usually be at most one minor version ahead of proxies.	Skipping minor versions.
Rollback	Rolling back is relabeling the namespace to the old revision and restarting the workloads.	Upgrading in place without a rollback path.
Monitor	Request success rate and p99 latency by service.	Upgrading during peak traffic.

Flow: Revisions to Skew and order / Skew and order to Rollback / Rollback to Monitor

Incident: Proxies that could not read the new config

Incident: *An in-place upgrade left old proxies with unsupported configuration.*

```
istiod: pushing config to 812 proxies
proxy: unknown field in Listener
412 proxies rejected configuration (NACK)
```

The control plane was upgraded in place two versions at once, which broke compatibility. The team rolled back, moved to one-minor-version upgrades with revisions, and added a pre-upgrade check and a canary namespace to every plan.

Common mistakes

- Skipping minor versions
- Upgrading in place without a rollback path
- Not restarting workloads after changing revisions
- Upgrading during peak traffic

Check yourself

What does a revision allow?

Answer: Two control planes side by side

Which is upgraded first?

Answer: The control plane

How do you roll back?

Answer: Relabel to the old revision and restart

What should you watch during upgrade?

Answer: Error rate, latency and config push health

MARGA by EknathaLabs
marga.eknathalabs.com

Cheat sheet: Service mesh and Kubernetes networking

Check Services and endpoints, DNS, network policy, sidecars, sync and the response flags in proxy logs.

Service and endpoints

```
kubectl get svc,endpoints web -n shop
kubectl get svc web -n shop -o jsonpath='{.spec.selector}'
kubectl get pods -n shop --show-labels
kubectl get endpointslices -n shop
```

DNS from a pod

```
kubectl exec -n shop deploy/web -- cat /etc/resolv.conf
kubectl exec -n shop deploy/web -- nslookup db.shop
kubectl logs -n kube-system -l k8s-app=kube-dns --tail=20
```

Network policy

```
kubectl get networkpolicy -n shop
kubectl describe networkpolicy default-deny -n shop
kubectl exec -n shop deploy/web -- curl -s -m 2 http://db:5432
```

Tip: *A timeout where you expect a refusal often means a policy is dropping packets.*

Mesh health

```
istioctl version
istioctl proxy-status
istioctl analyze -n shop
kubectl get pods -n shop
kubectl label namespace shop istio-injection=enabled
kubectl rollout restart deployment -n shop
```

Proxy logs and flags

```
kubectl logs web-7d -c istio-proxy --tail=20
```

Flag	Meaning
UF	Upstream connection failure
UH	No healthy upstream
NR	No route configured
UC	Upstream connection terminated
URX	Retry limit exceeded

Method for a failing call

1. Find the first failing hop.
2. `istioctl proxy-status`: is the config SYNCED?
3. `istioctl analyze`: is the config valid?
4. Read the response flag.
5. Check endpoints, then policies (PeerAuthentication, AuthorizationPolicy).
6. Call the pod IP directly to take the mesh out of the picture.

Quick mesh checks

```
istioctl analyze -n shop
istioctl proxy-status
istioctl proxy-config clusters POD.shop | head
kubectl get peerauthentication,authorizationpolicy -A
```

MARGA by EknathaLabs marga.eknathalabs.com

Cautions

Danger: *Switching a namespace to STRICT mTLS rejects every client without a sidecar. Use PERMISSIVE first and check what is still plaintext.*

Warning: *Retries at several layers multiply load. Keep one layer of retries and set a budget, or a slowdown becomes an outage.*

Warning: *Weights in a VirtualService must add up to 100, and every subset must exist in a DestinationRule, or requests fail with 503.*

Caution: *A default-deny network policy blocks DNS too. Allow egress to kube-dns or every lookup will fail.*

MARGA by EknathaLabs
marga.eknathalabs.com